

ECE 220 Computer Systems & Programming

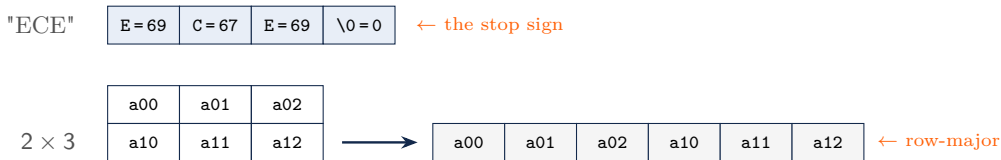
Lecture 10 — Strings; Multi-dimensional Arrays

September 24, 2026

Prof. Sayan Mitra mitras@illinois.edu Office hours: TBD
Course website: courses.grainger.illinois.edu/ece220/fa2026

The Representation Problem

How do you fit text and tables into a machine that only knows flat arrays of numbers?



Two conventions carry it all: the **terminating zero** and **row-major order**.

Today's Two Questions

**Arrays with a convention on top —
that is all strings and matrices are.**

1. How does C know where a string *ends*?
2. Where does `a[i][j]` actually live in flat memory?

But first: two pointer traces from Tuesday's material — one of them hides a crash.

Warm-up: Two Traces

```
int array[2] = {3, 5};
int *ptr = array;
int i;
for (i = 0; i < 2; i++, ptr++)
    *(ptr + 1) = *ptr + 1;
/* --- trace 2: final contents? ----- */
int buffer[4] = {1, 3, 5, 7};
int *p = buffer;
*(buffer + 1) = 9;
p[3] = 11;
for (i = 0; i < 4; i++)
    printf("%d ", buffer[i]);
```

Trace 1: what gets modified?

Trace 2: final `buffer`?

C never checks bounds. The compiler will cheerfully corrupt whatever lives next door — bounds are YOUR job.

Strings: char Arrays with a Stop Sign

```
char out[16] = "Result = "; /* + '\0' */
char in[20];
printf("%s(%d chars)\n", out, my_strlen(out));
```

...is exactly the same as

out[0]='R'; out[1]='e'; ...

out[8]=' '; out[9]='\0';

- A string is a `char` array whose end is marked by _____
- So "ECE 220" needs an array of at least _____ chars
- There is no separate string type — just the array and the convention

String I/O — and my_strlen

```
int my_strlen(char s[]) {          /* in-class */
    int n = 0;
    while (s[n] != '\0')
        n++;
    return n;
}
```

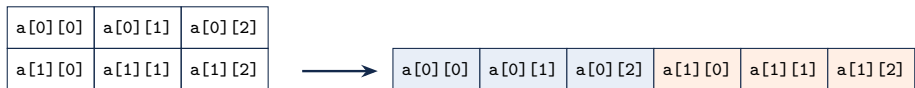
In-class: why does the loop need no size argument?

- `printf("%s", s)` — prints up to the _____
- `scanf("%s", s)` — reads until whitespace, appends the terminator
- Why no `&` on `s`?

scanf will happily read 50 chars into your 20-char array. Bounds, again, are your job.

Multi-dimensional Arrays

`int a[2][3];` — 2 rows, 3 columns, stored **row-major**:



- `a[i][j]` lives at flat offset _____ (N = number of columns)
- That is why a function parameter must state the column count: `int m[][N]`

Declaring and Initializing

Three equivalent initializations:

```
int a[2][3] = {{1,2,3}, {4,5,6}};
```

```
int a[][3] = {{1,2,3}, {4,5,6}};
```

(rows counted for you)

```
int a[2][3] = {1,2,3,4,5,6};
```

(flat, filled row-major)

In-class:

- Why is `int a[2][]` not allowed? _____
- Can we *reassign* with `{}` later? _____

In-Class Problem: Exchange Two Rows

`row_exchange(m, x, y)`: swap rows `x` and `y` of an $M \times N$ matrix.

- How many element swaps?

- Which loop variable — rows or columns? _____
- Reuse alert: this is Lecture 9's `swap` inside Lecture 6's loop

```
void row_exchange(int matrix[M][N], int row_x, int row_y) {  
    /* declare a column counter and a temporary */  
  
    /* walk the columns; swap the two rows' cells */  
  
}
```

Rows 0 and 2 exchanged: 9 10 11 12 / 5 6 7 8 / 1
2 3 4

Summary & What's Next

Today

- **Bounds are your job** — C writes past array ends without a word of protest
- **Strings**: char arrays ending in `'\0'`; `%s` I/O leans on the terminator; array names need no `&`
- **2-D arrays**: row-major; `a[i][j]` is flat offset $i \times N + j$ — which is why parameters must declare the column count

Next lecture

- Problem solving with pointers and arrays — and Midterm 1 is Thursday, October 1

Code from today: `warmup.c`, `strings.c`, `rowswap.c`.